

ステップ4：クラスとエクステンドを理解する

a. Flutterにおけるクラスとは？

- クラスはオブジェクト（Flutterではウィジェット）を作るための設計図.
- Flutterアプリは,クラスとして定義されたカスタムウィジェットで構築される.
- 各ウィジェットクラスは,UIの外観と動作を定義する.
- Flutterでウィジェットクラスを定義する一般的な方法はこちら：

```
class MyApp extends StatelessWidget {
```

▪ 説明:

- ◇ class MyApp → MyApp という名前のウィジェットを定義する.
- ◇ extends StatelessWidget → MyApp は StatelessWidget を継承しており,動的に変化しないウィジェットである.

b. Flutterにおける「extends」の意味とは？

- extends は,あるクラスを継承し,新しいクラスを作るためのキーワード.
- extends StatelessWidget を使うと,ウィジェットが変更されない（ステートレス）.
- ウィジェットは,親ウィジェットがリビルドしない限り更新されない.
- これは Stateless ウィジェットの完全な例：

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        appBar: AppBar(title: Text('My Flutter App')),  
        body: Center(child: Text('Hello, Flutter!')),  
      ),  
    );  
  }  
}
```

▪ 説明:

- ◇ class MyApp extends StatelessWidget → MyApp はステートを持たないウィジェット.
- ◇ build() メソッド → ウィジェットの UIを定義する.
- ◇ MaterialApp → アプリのルートウィジェット.

c. super.key とは何か,なぜ使うのか？

- super.key は,StatelessWidget や StatefulWidget などの親クラスに key を渡すために使われる.
- Flutter が ウィジェットツリー内のウィジェットを効率的に識別・更新するのに役立つ.

- 必須ではありませんが,ウィジェットの最適化や再利用性を高めるために 推奨される.

d. Java との比較

- Dart も Java も `extends` を使ってサブクラスを作る.
- Dartの方がシンプルで,Flutterのウィジェット設計に最適化されている.
- Javaでは通常,UIは別ファイルで定義されるが,Flutterではウィジェットクラス内に直接書く.

```
public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

ステップ5 : `@override` を使ったメソッドのオーバーライド

a. Flutterにおける`@override`とは？

- `@override` は,子クラスのメソッドが親クラスのメソッドを置き換えるときに使う.
- Flutterでは,`@override` は `StatelessWidget` や `StatefulWidget` から継承された `build()` によく使用される.
- 親クラスのメソッドを正しくオーバーライドしていることを保証する.
- `build()`メソッドをオーバーライドする例：

```
@override
Widget build(BuildContext context) {
```

- 説明:
 - ◇ `@override` → この `build()` メソッドが親クラスの `build()` を置き換えていることをFlutterに伝える.
 - ◇ `build(BuildContext context)` → UIの表示方法を定義する.
 - ◇ `@override` がなくても Dartは動作するが,コードの明確性を向上させ,ミスを防ぐのに役立つ.

b. なぜ`@override`が重要なのか？

- メソッドが正しくオーバーライドされていることを保証する.
- コードを読みやすくする → このメソッドが親クラスのメソッドを置き換えていることを明示的に示す.
- 意図しないミスを防ぐ → `@override` がないと,メソッド名を間違えたときに Dartがエラーを検出しない可能性がある.

c. 比較：Java と Flutter における`@override`

- Java も Flutter (Dart) も `@override` を使って,メソッドのオーバーライドを明示する.